

1.7 Projection



Core skills you will master in this section:

- ▶ Scalar projection: a vector's "coordinate value" along a given direction.
- ▶ Vector projection: "scalar projection $\times$ unit vector," representing the orthogonal component along the projection direction.
- ▶ Orthogonal complement: the part left over after orthogonal projection, representing the component that cannot be linearly expressed by the direction vector.
- ▶ Orthogonal decomposition: splitting a vector into two mutually perpendicular components, representing the projection and the orthogonal complement.
- ▶ Computing coordinates: using scalar projection to compute any vector's coordinate values in a rotated Cartesian coordinate system.

In the previous section, when discussing the vector inner product, we mentioned the concept of scalar projection. Scalar projection can be understood as the "length" of a vector's orthogonal projection onto the direction of another vector — that is, its "coordinate value"; scalar projection is an important application of the inner product. This section also introduces vector projection, which amounts to "coordinate value $\times$ direction vector." We'll also cover the orthogonal complement, which represents the part of a vector that cannot be linearly expressed by the projection direction and is orthogonal to that direction. Finally, this section explores the concept of vector decomposition — splitting a vector into multiple components — and introduces the standard orthonormal basis.

Scalar Projection

As shown in Figure 1 (a), given nonzero vectors $\mathbf{a}$ and $\mathbf{b}$, the angle between them is θ .

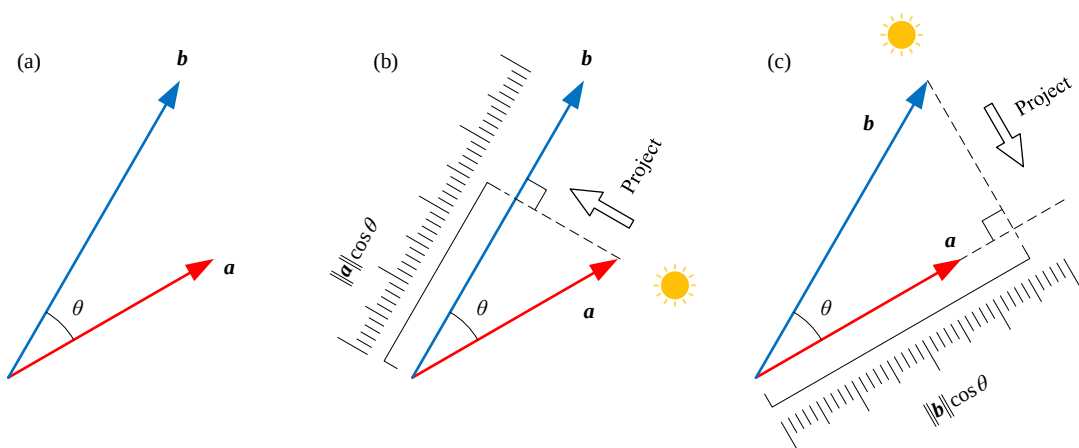


Figure 1. Scalar Projection

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

As shown in Figure 1 (b), the scalar projection of $\mathbf{a}$ onto the direction of $\mathbf{b}$ is

$$\|\mathbf{a}\| \cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|} = \mathbf{a} \cdot \frac{\mathbf{b}}{\|\mathbf{b}\|} = \mathbf{a} \cdot \hat{\mathbf{b}}$$

Here, $\hat{\mathbf{b}}$ is the unit vector (direction vector) of the nonzero vector $\mathbf{b}$.

As the name suggests, the result of a scalar projection is a scalar, so it carries no direction information.

Scalar projection can be understood as the “coordinate” of a given vector's orthogonal projection along a specific direction.

For example, in Figure 1 (b), $\|\mathbf{a}\| \cos \theta$ represents the “coordinate” of the orthogonal projection of vector $\mathbf{a}$ onto the direction of vector $\mathbf{b}$ ($\hat{\mathbf{b}}$); as shown in Figure 1 (b), orthogonal projection means the dashed line in the figure is perpendicular to vector $\mathbf{b}$.

As shown in Figure 1 (c), conversely, the scalar projection of $\mathbf{b}$ onto the direction of $\mathbf{a}$ is

$$\|\mathbf{b}\| \cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|} = \frac{\mathbf{a}}{\|\mathbf{a}\|} \cdot \mathbf{b} = \hat{\mathbf{a}} \cdot \mathbf{b}$$

Here, $\hat{\mathbf{a}}$ is the unit vector (direction vector) of the nonzero vector $\mathbf{a}$.

In other words, $\|\mathbf{b}\| \cos \theta$ is the “coordinate” of the orthogonal projection of vector $\mathbf{b}$ onto the direction of vector $\mathbf{a}$ ($\hat{\mathbf{a}}$). As shown in Figure 1 (c), the dashed line is perpendicular to vector $\mathbf{a}$.

For example, given the vector $\mathbf{a} = [3, 4]^T$, the scalar projection of $\mathbf{a}$ onto the direction of $\mathbf{e}_1$ is

$$\mathbf{a} \cdot \mathbf{e}_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 3$$

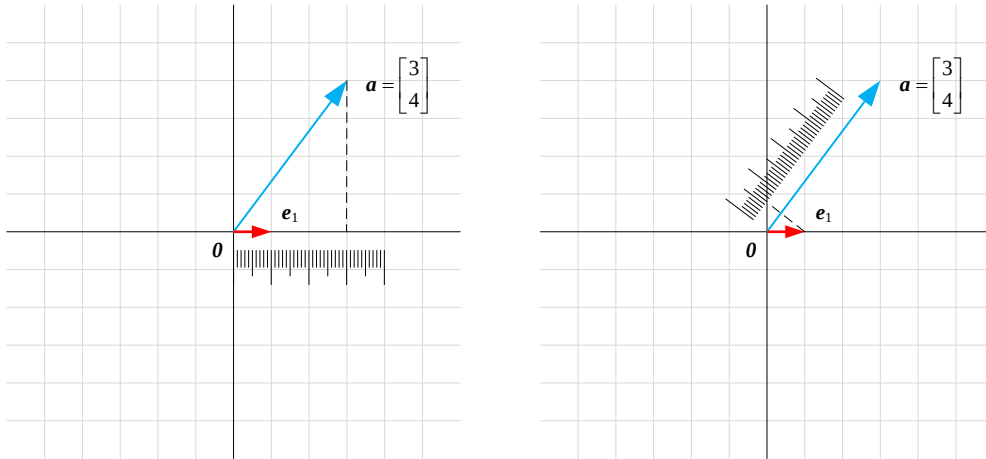
We can compute the scalar projection directly with the vector inner product because $\mathbf{e}_1$ is a unit vector.

Conversely, we compute the scalar projection of $\mathbf{e}_1$ onto the direction of $\mathbf{a}$.

$$\mathbf{e}_1 \cdot \hat{\mathbf{a}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} = 3/5$$



As an exercise, compute the mutual scalar projections of vectors $\mathbf{a}$ and $\mathbf{e}_2$ onto each other.

Figure 2. Mutual Scalar Projections Between Vectors $\mathbf{a}$ and $\mathbf{e}_1$

Code 1 completes the scalar-projection computation above. Let's walk through the key lines below.

- a** creates two 1D arrays, representing the vectors $\mathbf{a}$ and $\mathbf{e}_1$.
- b** computes the dot product of $\mathbf{a}$ and $\mathbf{e}_1$, and of $\mathbf{e}_1$ and $\mathbf{a}$, separately — see if you can tell whether the results are the same.
- c** uses `numpy.linalg.norm()` to compute the lengths of the vectors $\mathbf{a}$ and $\mathbf{e}_1$.
- d** computes the scalar projection of $\mathbf{a}$ onto the direction of $\mathbf{e}_1$ — that is, the inner product of $\mathbf{a}$ and $\mathbf{e}_1$ divided by the length of $\mathbf{e}_1$.
- e** computes the scalar projection of $\mathbf{e}_1$ onto the direction of $\mathbf{a}$ — that is, the inner product of $\mathbf{a}$ and $\mathbf{e}_1$ divided by the length of $\mathbf{a}$.

Code 1. Computing Scalar Projection |  LA_01_07_01.ipynb

```

## 初始化
import numpy as np

## 定义向量
a a = np.array([3, 4])
    e1 = np.array([1, 0])

## 计算点积
b dot_a_e1 = np.dot(a, e1)
    dot_e1_a = np.dot(e1, a)

## 计算向量模长
c norm_a = np.linalg.norm(a)
    norm_e1 = np.linalg.norm(e1)

## 计算标量投影
d scalar_proj_a_on_e1 = dot_a_e1 / norm_e1 # a 在 e1 上的投影
e scalar_proj_e1_on_a = dot_e1_a / norm_a # e1 在 a 上的投影
  
```

Scalar Projection of RGB Colors

Imagine a beam of yellow light passing through a red filter, leaving only the intensity of red light. This intensity value can be thought of as the scalar projection of the yellow light onto the red axis.

As shown in Figure 3 (a), in RGB space, the yellow-light vector is $[1, 1, 0]^T$, and the red vector corresponding to the red filter is $[1, 0, 0]^T$; the scalar projection of the yellow light onto the red-light vector is

$$\frac{[1 \ 1 \ 0]^T \cdot [1 \ 0 \ 0]^T}{\|[1 \ 0 \ 0]^T\|} = 1$$

This scalar projection can be understood as the yellow light's "coordinate" along the red direction – that is, the intensity of red light. This shows that yellow light contains 100% of the intensity of red light.

Conversely, as shown in Figure 3 (b), the scalar projection of the red light onto the yellow light is

$$\frac{[1 \ 1 \ 0]^T \cdot [1 \ 0 \ 0]^T}{\|[1 \ 1 \ 0]^T\|} = \frac{\sqrt{2}}{2}$$

The yellow-light vector has length $\sqrt{2}$, of which red light accounts for $1/2$.



Similarly, as an exercise, compute the mutual scalar projections of magenta and blue light.

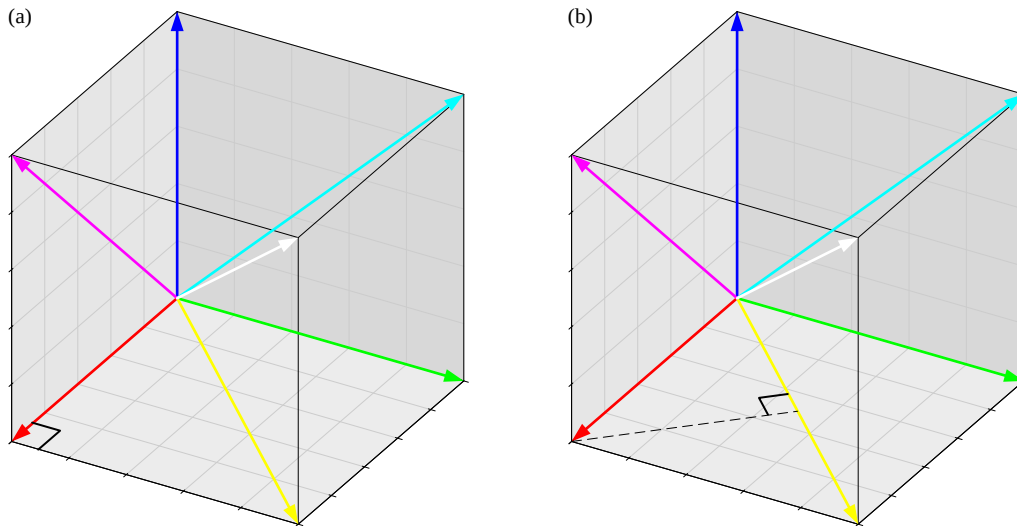


Figure 3. Mutual Scalar Projections of Red and Yellow Light

Meanwhile, when cyan $[0, 1, 1]^T$ passes through the red filter, no light gets through at all – the reason is simple: cyan light has no red component whatsoever. Scalar projection lets us verify this:

$$\frac{[0 \ 1 \ 1]^T \cdot [1 \ 0 \ 0]^T}{\|[1 \ 0 \ 0]^T\|} = 0$$

Put another way, any vector in the blue-green plane has a scalar projection of 0 onto the red direction.

? As an exercise, compute the scalar projections of blue light and green light onto the red direction.

Now imagine a beam of white light passing through a red filter, leaving only red light — this is equivalent to the scalar projection of white light onto the red direction.

In RGB space, the white-light vector is $[1, 1, 1]^T$, and the red vector is $[1, 0, 0]^T$; the scalar projection of white light onto the red-light vector is

$$\frac{[1 \ 1 \ 1]^T \cdot [1 \ 0 \ 0]^T}{\|[1 \ 0 \ 0]^T\|} = 1$$

This value can be understood as white light's “coordinate” along the red axis — that is, the intensity of red light. This shows that white light contains 100% of the intensity of red light.

Conversely, the scalar projection of red light onto white light is

$$\frac{[1 \ 1 \ 1]^T \cdot [1 \ 0 \ 0]^T}{\|[1 \ 1 \ 1]^T\|} = \frac{\sqrt{3}}{3}$$

The white vector has length $\sqrt{3}$, of which red light accounts for $1/3$.

To better illustrate the projection, Figure 4 uses two different viewing angles.

? As an exercise, compute the mutual scalar projections between the white and blue vectors, and between the white and green vectors.

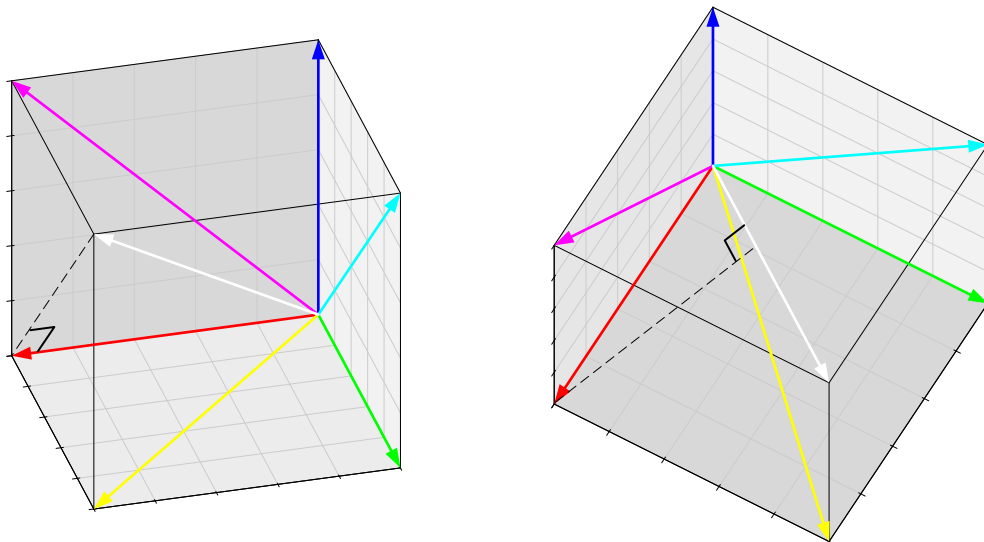


Figure 4. Mutual Scalar Projections of Red and White Light

Vector Projection

From what we've covered so far in this section, you know that scalar projection tells us how large a vector is along a specified direction — its “coordinate” — much like measuring the length of an object's shadow along a line.

Vector projection, on the other hand, amounts to “coordinate value $\times$ direction vector”; this direction vector is the unit vector of the projection direction.

As shown in Figure 5 (a), given nonzero vectors $\mathbf{a}$ and $\mathbf{b}$, the vector projection of $\mathbf{a}$ onto the direction of $\mathbf{b}$ is

$$\text{proj}_{\mathbf{b}} \mathbf{a} = \underbrace{\|\mathbf{a}\| \cos \theta}_{\text{Scalar projection}} \times \underbrace{\hat{\mathbf{b}}}_{\text{Direction vector}} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|} \times \frac{\mathbf{b}}{\|\mathbf{b}\|} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|^2} \mathbf{b} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{b} \cdot \mathbf{b}} \mathbf{b}$$

$\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|}$ is the scalar projection of $\mathbf{a}$ onto the direction of $\mathbf{b}$ — that is, the coordinate value; $\frac{\mathbf{b}}{\|\mathbf{b}\|}$ is the direction vector of $\mathbf{b}$ — that is, its unit vector, the normalized vector.

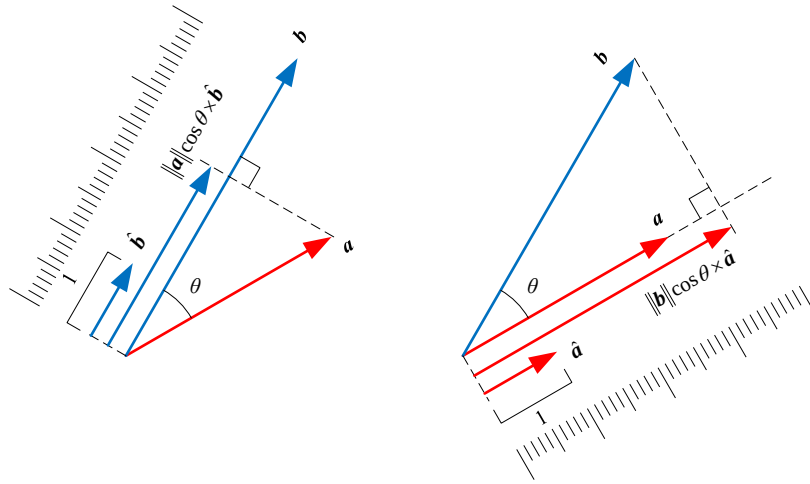


Figure 5. Vector Projection

Conversely, as shown in Figure 5 (b), the vector projection of $\mathbf{b}$ onto the direction of $\mathbf{a}$ is

$$\text{proj}_{\mathbf{a}} \mathbf{b} = \underbrace{\|\mathbf{b}\| \cos \theta}_{\text{Scalar projection}} \times \underbrace{\hat{\mathbf{a}}}_{\text{Direction vector}} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|} \times \frac{\mathbf{a}}{\|\mathbf{a}\|} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|^2} \mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}} \mathbf{a}$$

Similarly, $\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|}$ is the scalar projection of $\mathbf{b}$ onto the direction of $\mathbf{a}$ (the coordinate value); $\frac{\mathbf{a}}{\|\mathbf{a}\|}$ is the direction vector (unit vector) of $\mathbf{a}$.

In summary, scalar projection gives only the “coordinate” along the projection direction, while vector projection, by introducing the direction vector, specifies both the “coordinate and direction” of the projection.



LA_01_06_02.ipynb explains how to compute vector projection; please study it on your own.

Vector Projection of RGB Colors

In RGB space, the red-light vector $[1, 0, 0]^T$ is itself a unit vector; earlier we already computed the scalar projection of yellow light onto red light.

Therefore, the vector projection of the yellow-light vector $[1, 1, 0]^T$ onto the red-light vector is

$$1 \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Conversely, the unit vector of yellow light is

$$\frac{1}{\sqrt{2}} \times \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

Earlier we computed that the scalar projection of red light onto yellow light is $\frac{\sqrt{2}}{2}$; this makes it easy to compute the vector projection of red light onto yellow light:

$$\frac{\sqrt{2}}{2} \times \frac{1}{\sqrt{2}} \times \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \frac{1}{2} \times \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

 As an exercise, compute the vector projection of white light onto the red direction; also compute the vector projection of red light onto the white direction.

Orthogonal Complement

Figure 3 (a) shows that after the yellow vector is orthogonally projected onto the red vector, a “remainder” is left over – this remainder is the green vector.

This remaining green vector is perpendicular to the red vector. In other words, the green vector represents the part of the yellow vector that cannot be expressed using the red vector.

In vector projection, this green vector also has its own name – the orthogonal complement (orthogonal rejection); it is also called the orthogonal remainder, the perpendicular component, or the normal component, emphasizing its orthogonal relationship to the projection direction.

Simply put, if a vector is orthogonally projected onto a given direction, the leftover part that cannot be expressed by that direction is the orthogonal complement with respect to that direction.

Given nonzero vectors $\mathbf{a}$ and $\mathbf{b}$, the orthogonal complement of $\mathbf{a}$ with respect to the direction of $\mathbf{b}$ is

$$\text{oproj}_{\mathbf{b}} \mathbf{a} = \mathbf{a} - \text{proj}_{\mathbf{b}} \mathbf{a}$$

$\text{oproj}_{\mathbf{b}} \mathbf{a}$ is the component of vector $\mathbf{a}$ that cannot be linearly expressed by $\mathbf{b}$. Clearly, $\text{oproj}_{\mathbf{b}} \mathbf{a}$ is perpendicular to $\mathbf{b}$.

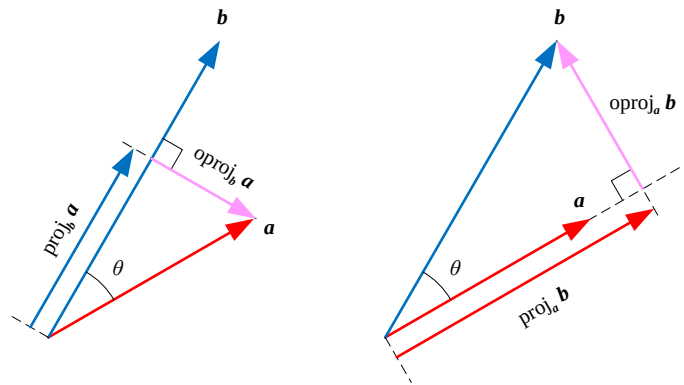


Figure 6. Orthogonal Complement

Conversely, the orthogonal complement of $\mathbf{b}$ with respect to the direction of $\mathbf{a}$ is

$$\text{oproj}_a \mathbf{b} = \mathbf{b} - \text{proj}_a \mathbf{b}$$

$\text{oproj}_a \mathbf{b}$ is the component of vector $\mathbf{b}$ that cannot be linearly expressed by $\mathbf{a}$. Clearly, $\text{oproj}_a \mathbf{b}$ is perpendicular to $\mathbf{a}$.

Orthogonal Decomposition of a Vector

The core idea of vector decomposition is to split a vector into two or more components whose sum still equals the original vector.

In fact, vector decomposition is nothing new to you. As mentioned earlier, in the Cartesian coordinate system, every vector can be expressed as the sum of two components along the coordinate axes.

We can write $\mathbf{x} = [x_1, x_2]^T$ as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = x_1 \mathbf{e}_1 + x_2 \mathbf{e}_2$$

For example, the vector $\mathbf{a} = [3, 4]^T$ can be written as

$$\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 4 \end{bmatrix} = 3 \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 4 \times \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 3\mathbf{e}_1 + 4\mathbf{e}_2$$

As shown in Figure 7, the horizontal component — the part along the x_1 axis — is $[3, 0]^T$, which can be written as $3\mathbf{e}_1$.

The vertical component — the part along the x_2 axis — is $[0, 4]^T$, which can be written as $4\mathbf{e}_2$. This is clearly an orthogonal decomposition of a vector.

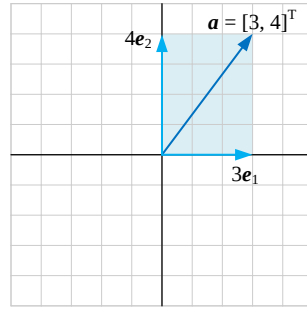
Figure 7. Decomposition of Vector $\mathbf{a} = [3, 4]^T$ in $[\mathbf{e}_1, \mathbf{e}_2]$

Figure 7 shows what is actually the orthogonal decomposition of a vector. Orthogonal decomposition splits a vector into two or more mutually orthogonal components, and this way of decomposing ensures the components don't interfere with one another. In Euclidean space, the existence and uniqueness of an orthogonal decomposition depend on how the inner product is defined, which is why it has such wide applications across mathematics, physics, and engineering.

$\mathbf{e}_1$ and $\mathbf{e}_2$ form the standard orthonormal basis (standard basis, natural basis), denoted $[\mathbf{e}_1, \mathbf{e}_2]$.

The standard orthonormal basis is a set of basis vectors in a vector space, commonly used to represent the positive directions of the coordinate axes in Euclidean space.

Let's now unpack the term "standard orthonormal basis" piece by piece.

Let's start with basis. A basis is a set of linearly independent vectors in a vector space, whose linear combinations can represent any vector in that space.

Each vector in a basis is called a basis vector. Basis vectors are like the skeleton that supports a space's grid structure.

As mentioned earlier in this book, linear independence means that no vector in a set can be expressed as a linear combination of the others — that is, these vectors are directionally independent of one another and cannot be substituted for each other.

As also mentioned earlier in this book, a linear combination is the operation of multiplying a set of vectors by scalar coefficients and adding the results to produce a new vector. For example, $\mathbf{e}_1$ and $\mathbf{e}_2$, through the linear combination $(x_1\mathbf{e}_1 + x_2\mathbf{e}_2)$, can express any vector $\mathbf{x}$ in the plane.

The word "standard" in "standard orthonormal basis" relates to the "natural" or "standard" in natural basis or standard basis. It means this set of basis vectors is the most natural, most commonly used basis in the vector space, typically the unit vectors pointing in the positive directions of the coordinate axes. For example, the unit vector $\mathbf{e}_1$ points in the positive x_1 direction, and the unit vector $\mathbf{e}_2$ points in the positive x_2 direction.

It's fine if these vector-space concepts only feel vaguely familiar for now — later in the book we'll cover them systematically in their own dedicated discussion.

Rearranging, we can decompose the vector $\mathbf{a}$ into two parts:

$$\mathbf{a} = \text{proj}_{\mathbf{b}} \mathbf{a} + \text{oproj}_{\mathbf{b}} \mathbf{a}$$

As shown in Figure 8, $\text{proj}_{\mathbf{b}} \mathbf{a}$ and $\text{oproj}_{\mathbf{b}} \mathbf{a}$ — these two vector components are perpendicular to each other. The vector projection $\text{proj}_{\mathbf{b}} \mathbf{a}$ is parallel to vector $\mathbf{b}$, and the orthogonal complement $\text{oproj}_{\mathbf{b}} \mathbf{a}$ is perpendicular to vector $\mathbf{b}$.

Similarly, the vector $\mathbf{b}$ can be decomposed into two parts:

$$\mathbf{b} = \text{proj}_{\mathbf{a}} \mathbf{b} + \text{oproj}_{\mathbf{a}} \mathbf{b}$$

$\text{proj}_{\mathbf{a}} \mathbf{b}$ and $\text{oproj}_{\mathbf{a}} \mathbf{b}$ – these two vector components are perpendicular to each other. The vector projection $\text{proj}_{\mathbf{a}} \mathbf{b}$ is parallel to vector $\mathbf{a}$, and the orthogonal complement $\text{oproj}_{\mathbf{a}} \mathbf{b}$ is perpendicular to vector $\mathbf{a}$.

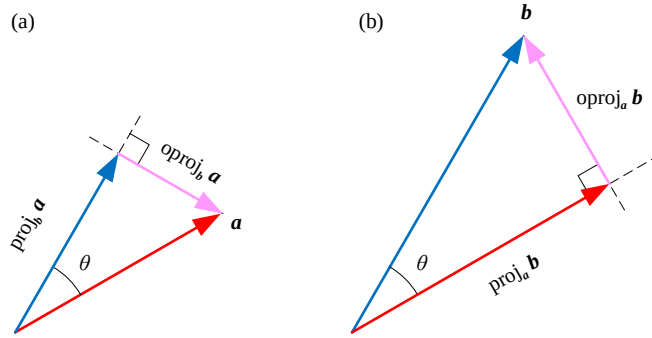


Figure 8. Orthogonal Vector Decomposition

In high school physics textbooks, the decomposition of force and velocity can use this same kind of orthogonal vector decomposition.

As shown in Figure 9 (a), an object on an inclined plane experiences the force of gravity; gravity can be decomposed into two components, one pointing down the slope and one perpendicular to the slope's surface.

As shown in Figure 9 (b), an object on the inclined plane moves down along the slope; its velocity can be decomposed into two components, one horizontal and one vertical.

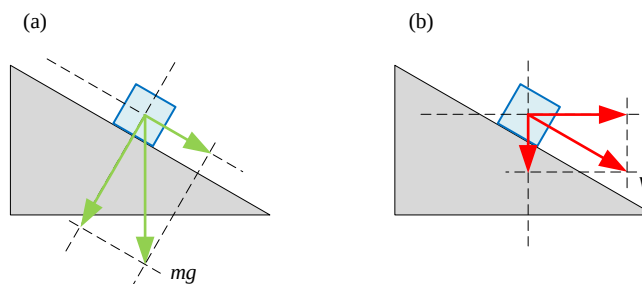


Figure 9. Orthogonal Decomposition of Force and Velocity

Computing Coordinates

In Figure 10 (a) – the familiar Cartesian coordinate system – the vector $\mathbf{a} = [4, 3]^T$ has coordinates (4, 3).

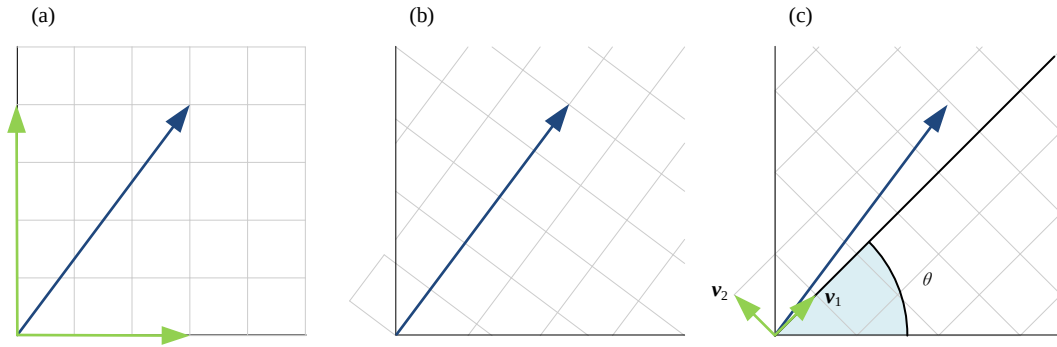


Figure 10. Orthogonal Decomposition of a Vector

As shown in Figure 10 (b), the vector $\mathbf{a} = [4, 3]^T$ can be orthogonally decomposed as

$$\begin{bmatrix} 4 \\ 3 \end{bmatrix} = 5 \times \begin{bmatrix} 4/5 \\ 3/5 \end{bmatrix} + 0 \times \begin{bmatrix} -3/5 \\ 4/5 \end{bmatrix}$$

This can be understood as follows: in the coordinate system described by this “rotated square grid,” the vector $\mathbf{a} = [4, 3]^T$ has coordinates (5, 0).

In fact, the plane has infinitely many such “rotated square grids”; we can find infinitely many sets of orthonormal vectors that satisfy

$$\mathbf{v}_1 = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}$$

As an exercise, compute the lengths of $\mathbf{v}_1$ and $\mathbf{v}_2$ to verify whether both equal 1; also verify that the inner product of $\mathbf{v}_1$ and $\mathbf{v}_2$ is 0.

In other words, as θ takes different values, we get different “rotated square grids”; each of these grids can serve as a “Cartesian” coordinate system that describes $\mathbf{a} = [4, 3]^T$.

This raises a question: how do we obtain the coordinates in these different “rotated square grids”?

This is where the scalar projection introduced earlier in this section comes in.

For example, the scalar projection of $\mathbf{a} = [4, 3]^T$ onto $\mathbf{v}_1$ is

$$\frac{\mathbf{a} \cdot \mathbf{v}_1}{\|\mathbf{v}_1\|} = \mathbf{a} \cdot \mathbf{v}_1 = \begin{bmatrix} 4 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} = 4 \cos \theta + 3 \sin \theta$$

This value is the coordinate of $\mathbf{a}$ along $\mathbf{v}_1$.

Similarly, the scalar projection of $\mathbf{a} = [4, 3]^T$ onto $\mathbf{v}_2$ is

$$\frac{\mathbf{a} \cdot \mathbf{v}_2}{\|\mathbf{v}_2\|} = \mathbf{a} \cdot \mathbf{v}_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} \sin \theta \\ -\cos \theta \end{bmatrix} = 4 \sin \theta - 3 \cos \theta$$

$\mathbf{a} = [4, 3]^T$'s vector component (vector projection) along $\mathbf{v}_1$ is

$$(4 \cos \theta + 3 \sin \theta) \cdot \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}$$

$\mathbf{a} = [4, 3]^T$'s vector component (vector projection) along $\mathbf{v}_2$ is

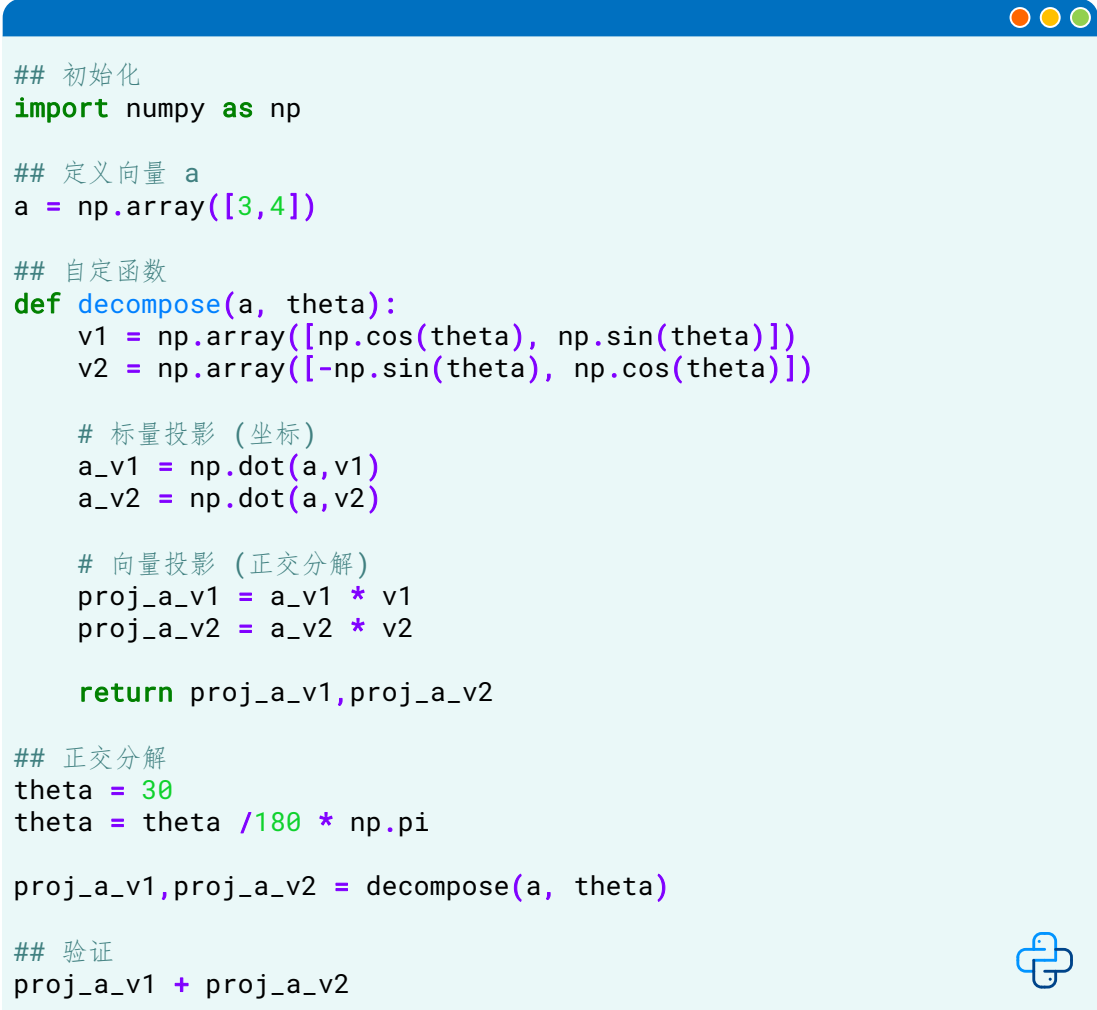
$$(4\sin\theta - 3\cos\theta) \cdot \begin{bmatrix} \sin\theta \\ -\cos\theta \end{bmatrix}$$

The sum of these two components is

$$(4\cos\theta + 3\sin\theta) \cdot \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix} + (4\sin\theta - 3\cos\theta) \cdot \begin{bmatrix} \sin\theta \\ -\cos\theta \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

Code 2 shows how to complete an orthogonal decomposition of a vector.

Code 2. Orthogonal Decomposition of a Vector |  LA_01_07_03.ipynb



```
## 初始化
import numpy as np

## 定义向量 a
a = np.array([3, 4])

## 自定义函数
def decompose(a, theta):
    v1 = np.array([np.cos(theta), np.sin(theta)])
    v2 = np.array([-np.sin(theta), np.cos(theta)])

    # 标量投影 (坐标)
    a_v1 = np.dot(a, v1)
    a_v2 = np.dot(a, v2)

    # 向量投影 (正交分解)
    proj_a_v1 = a_v1 * v1
    proj_a_v2 = a_v2 * v2

    return proj_a_v1, proj_a_v2

## 正交分解
theta = 30
theta = theta / 180 * np.pi

proj_a_v1, proj_a_v2 = decompose(a, theta)

## 验证
proj_a_v1 + proj_a_v2
```

Let's walk through the key lines in Code 2.

a defines a function named `decompose`, meaning “decompose,” which takes two input parameters: a vector `a` and an angle `theta` (in radians). The function's job is to split vector `a` into components along two directions.

b uses the angle `theta` to construct a unit vector $\mathbf{v}_1$, pointing in the direction obtained by rotating counterclockwise by `theta`. `np.cos` and `np.sin` compute the cosine and sine respectively, giving the horizontal and vertical components. This vector has length 1, and represents the first direction vector we want.

then constructs the unit vector for the second direction, $\mathbf{v}_2$, which is perpendicular to $\mathbf{v}_1$ — that is, the two directions are orthogonal to each other.

c computes the size of vector $\mathbf{a}$'s projection onto the direction of the unit vector (direction vector) $\mathbf{v}_1$ — called the scalar projection — using the `np.dot` function, which represents the “dot product” or “inner product.” The result of the dot product is a number representing “how much” of $\mathbf{a}$ lies along the direction of $\mathbf{v}_1$.

then computes the size of vector $\mathbf{a}$'s projection onto the direction of the unit vector (direction vector) $\mathbf{v}_2$ — again, the scalar projection.

d multiplies the scalar projection's size by the unit direction $\mathbf{v}_1$, giving a new vector `proj_a_v1`, which represents the component of vector $\mathbf{a}$ along $\mathbf{v}_1$. This step combines “size” and “direction” into a single vector.

then computes the component of vector $\mathbf{a}$ along $\mathbf{v}_2$.

e calls the `decompose` function defined earlier, passing in vector $\mathbf{a}$ and angle θ as arguments. After running, the function returns two vectors: $\mathbf{a}$'s projection onto the first direction, and its projection onto the perpendicular direction.

f Finally, we add these two projection vectors together to get a new vector. If the decomposition above is correct, the sum of these two projections should equal the original vector $\mathbf{a}$.

Decomposing White Light

Back in RGB space, Figure 11 shows three ways to orthogonally decompose the white-light vector.

As shown in Figure 11 (a), the white-light vector can be orthogonally decomposed into yellow light and blue light; clearly, blue is perpendicular to yellow, and blue is also orthogonal to the x_1x_2 plane.

Similarly, the white-light vector can also be orthogonally decomposed into magenta light and green light, as shown in Figure 11 (b); green is perpendicular to magenta, and green is orthogonal to the x_1x_3 plane. The white-light vector can also be decomposed into cyan light and red light, as shown in Figure 11 (c); red is perpendicular to cyan, and red is orthogonal to the x_2x_3 plane.

In fact, decomposing white light into “red + green + blue” is also an orthogonal decomposition; red, green, and blue — these three vector components — are mutually orthogonal.

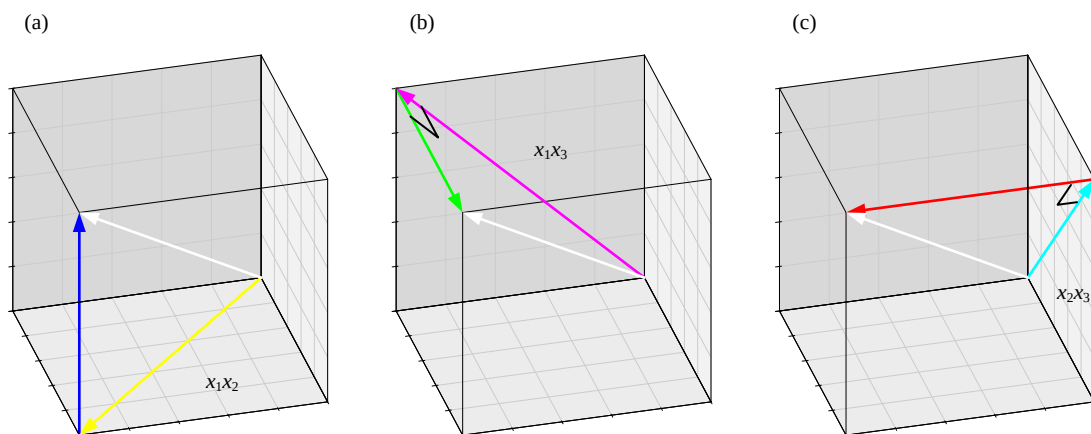


Figure 11. Three Decompositions of White Light

Not Just Axis-Aligned Unit Square Grids

Figure 12 shows several different ways to decompose the vector $\mathbf{a}$. Some grids are rectangular, some are rotated squares, and others are general parallelograms. Despite their different shapes, these grids all share the same traits – parallel, evenly spaced, and passing through the origin.

Clearly, there are infinitely many ways to decompose the vector $\mathbf{a}$, conversely, there are infinitely many different types of grids (bases) that can tile the entire plane.

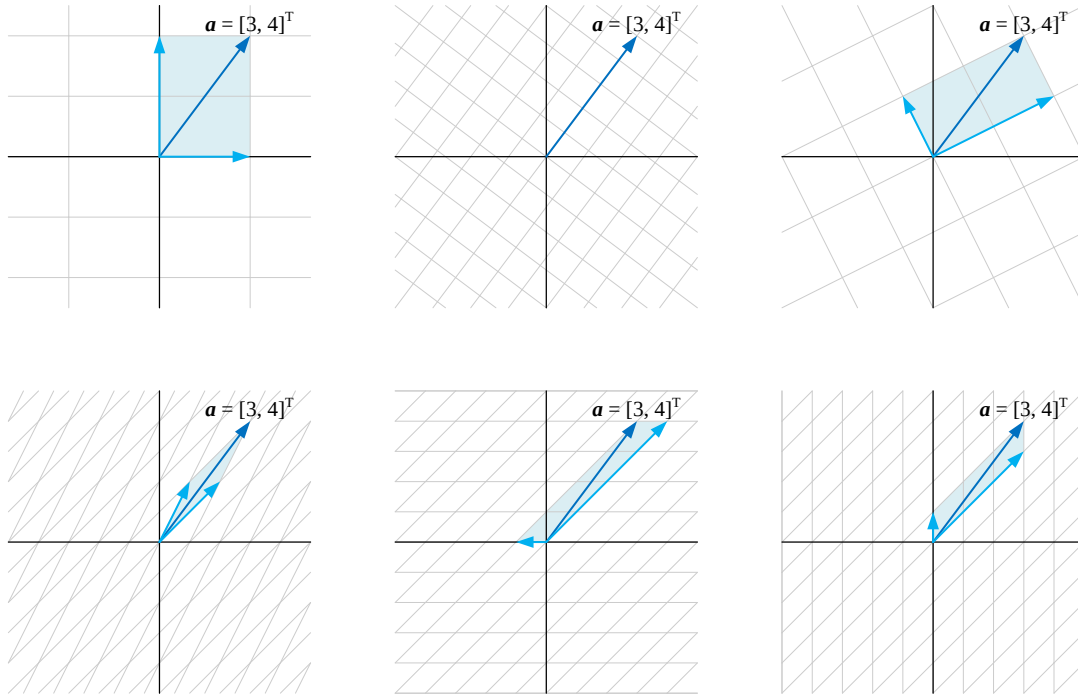


Figure 12. Different Ways to Decompose the Vector $\mathbf{a} = [3, 4]^T$

As shown in Figure 13, in RGB space we can find nearly countless orthogonal and non-orthogonal decompositions of white light. We first randomly generate a color vector in RGB space, then compute the difference between the white vector and this color vector; drawing these two color vectors tip-to-tail using the triangle rule then recovers the white-light vector.

Vector decomposition can be understood as the inverse operation of vector addition: splitting a vector into two or more components whose sum equals the original vector.

In other words, if vector addition combines multiple vectors into a single whole, then vector decomposition, given the result, finds the components that satisfy that additive relationship.

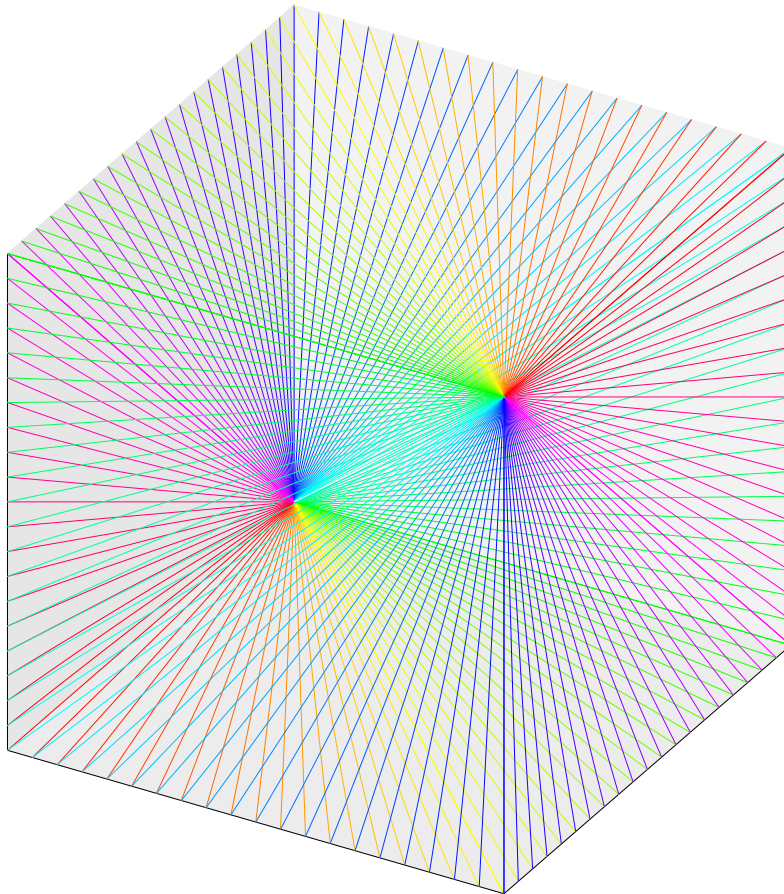


Figure 13. More Orthogonal and Non-Orthogonal Decompositions of White Light



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Write code to compute the scalar projection of the vector $\mathbf{a} = [3, 4]^T$ onto $[1, 1]$, and verify it by hand.

Q2. Write code to compute the vector projection of the vector $\mathbf{a} = [3, 4]^T$ onto $[-1, 1]$, and verify it by hand.

Q3. Given the vector $\mathbf{a} = [3, 4]^T$ and $\mathbf{b} = [-1, 0]$, decompose the vector $\mathbf{a}$ into

- the component parallel to $\mathbf{b}$
- the component perpendicular to $\mathbf{b}$

Q4. Try writing out 10 vector decompositions similar to Figure 12.